

Lecture 18

Data Visualization with seaborn

Byeong-Hak Choe

bchoe@geneseo.edu

SUNY Geneseo

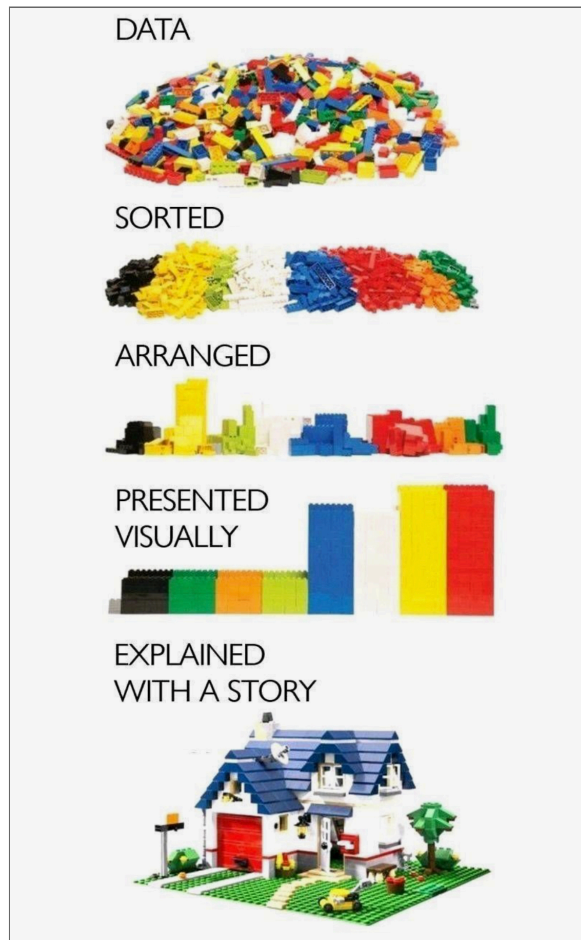
November 12, 2025

Exploratory Data Analysis

Exploratory Data Analysis (EDA)

- EDA combines **descriptive statistics, data transformation, and data visualization** to help you:
 1. **Generate meaningful questions** about your data
 2. **Explore answers** through visualizations, transformations, and descriptive statistics
 3. **Refine your understanding** by iterating on your questions and exploring new ones
- While descriptive statistics and data transformations are useful for exploring data, **visualizations often reveal patterns in data more clearly.**

Data Visualization



- **Graphs and charts** let us explore and learn about the structure of the information we have in **DataFrame**.
- Good data visualizations make it easier to communicate our ideas

and findings to other people.

Key Points in Data Visualization

- A graphic should display as much information as it can, with **the lowest possible cognitive strain to the viewer**.
- Strive for **clarity**.
 - Make the data stand out.
 - **Avoid too many superimposed elements** (e.g., too many curves in the same graphing space).
 - Avoid having the data all skewed to one side or the other of your graph.
- Visualization is an **iterative process**.
 - We should try making data visualization informative as much as we can.

Categorical vs. Continuous Variables

<u>Categorical</u>	<u>Continuous</u>	<u>Integer</u>
---------------------------	--------------------------	-----------------------

- A **categorical variable** is a variable whose value is obtained by counting.
 - Number of red marbles in a jar
 - Number of heads when flipping three coins
 - Students' letter grade
 - US state/county
- Categorical variables should be `category` or `string` (`object` in `DataFrame`).
- A **continuous variable** is a variable whose value is obtained by measuring and can have a decimal or fractional value.
 - Height/weight of students
 - Time it takes to get to school
 - Fuel efficiency of a vehicle (e.g., miles per gallon)
- Continuous variables should be `float`.
 - We can use `astype('float')` to make a variable `float`.
- For data visualization, `integer`-type variables could be treated as **either categorical or continuous**, depending on the context of analysis.

- If the values of an integer-type variable means **an intensity or an order**, the integer variable could be treated as **continuous**.
 - A variable of **age** integers (18, 19, 20, 21, ...) could be continuous.
 - A variable of integer-valued **MPG** (27, 28, 29, 30, ...) could be continuous.
- Otherwise, the integer variable is **categorical**.
 - A variable of **month** integers (1, ,2, ..., 12) is categorical.
- We can use `astype('int')` to make a variable `int`.

Making Discoveries from Visualization

Distribution

Relationship

Time Trend

Types of Viz

- From the distribution plots, we want to see the **variation**, the tendency of the values of a variable to change.
 - Which values are the most common? Why?
 - How *normal* is the data? How many peaks are there in the distribution?
 - Which values are rare?
- From the plots with two or more variables, we want to see **co-variation**, the tendency for the values of two or more variables to vary together in a **related** way.
- What type of **co-variation** occurs between variables?
 - Are they *positively* associated?
 - Are they *negatively* associated?
 - Are there *no association* between them?
- From time-series plots, we want to see the **evolution** of a variable over time.
 - Is there an overall upward or downward **trend**?
 - Does the trend look **linear** or **non-linear**?

- Are there **seasonal** or **cyclical** patterns (e.g., yearly, weekly)?
- Do we observe any **outliers**, **shocks**, or **structural breaks** that merit closer inspection?
- A **distribution** of a variable
 - *Categorical* (e.g., **bar charts** and more)
 - *Continuous* (e.g., **histograms**, **box plots**, and more)
- A **relationship** between *two continuous* variables (e.g., **scatter plots**, **best fitting lines**)
- A **time trend** of a variable (e.g., **line plots**, **best fitting lines** and more)
- Exploring **how distributions, relationships, and time trends vary across categories** can reveal important patterns and insights.
- Please check out **general tips on data visualization** in the project page.

Getting started with **seaborn**

Getting started with **seaborn**



- **seaborn** is a Python data visualization library based on **matplotlib**.
 - It allows us to easily create beautiful but complex graphics using a simple interface.

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
```

Data Visualization Workflow with **seaborn**

Step 1

Step 2

Step 3

Step 4

Step 5

- Read the descriptions of variables in a **DataFrame** if available.
- Check the **unit of an observation**: Are all values in one single observation measured for:
 - One organization (e.g., university, company)?
 - One geographic unit (e.g., zip code, county, state, country)?
 - One person?
 - One person in a specific point of time (e.g., year, month, week)?
 - One geographic unit in a specific point of time?
 - One organization in a specific point of time?
- Prepare a **DataFrame** for visualization.
 - Adding new variables
 - Filtering observations
 - Making **DataFrame** longer/wider
 - Group operations
- Figure out whether variables of interests are **categorical** or **continuous**.

- Check what types of the variables are (e.g., `float`, `int`, `datetime64`, `object`, `category`).
- Use `astype(DTYPE)` if needed.
- Choose appropriate
 - Geometric objects (e.g., `sns.histplot`, `sns.scatterplot`)
 - Aesthetic mappings (`(x = , y = , hue = )`)
 - Faceting (`FacetGrid(DATA, row = , col = ).map(GEOMETRIC_OBJECT, VARIABLES)` or `col/row` parameters in the function of geometric object)
- Pay attention to the unit of `x` and `y` axes.
- Repeat the process until you are satisfied with the visualization output.

Datasets and Plot Styles in **seaborn**

- Let's get the names of **DataFrames** provided by the **seaborn** library:

```
1 sns.get_dataset_names()
```

- Matplotlib provides plot **styles**:

```
1 plt.style.available  
2 plt.style.use('STYLE_YOU_WANT_TO_USE')
```

- Let's use the **titanic** and **tips** DataFrames:

```
1 titanic = sns.load_dataset('titanic')  
2 tips = sns.load_dataset('tips')
```

Bar Chart with `sns.countplot()`

Example

hue

```
1 sns.countplot(data =  
2                   x = 's
```

- Mapping
 - `data`: DataFrame.
 - `x`: Name of a categorical variable (column) in DataFrame
- A bar chart is used to plot the frequency of the different categories.
 - It is useful to visualize how values of a **categorical variable** are distributed.
 - A variable is **categorical** if it can only take one of a small set of values.
- We use `sns.countplot()` function to plot a bar chart

```
1 sns.countplot(data =  
2                   x = 'se  
3                   hue = '
```


- Mapping
 - **hue**: Name of a categorical variable
- We can further break up the bars in the bar chart based on another categorical variable.
 - This is useful to visualize how the distribution of a categorical variable varies by another categorical variable.

Histogram with `sns.histplot()`

```
1 sns.histplot(data = t
2               x = 'ag
3               bins = 5
```

- Mapping
 - `bins`: Number of bins
 - `binwidth`: Width of each bin
- A histogram is a **continuous** version of a bar chart.
 - It is used to plot the frequency of the different values.
 - It is useful to visualize how values of a **continuous variable** are distributed.
- We use `sns.histplot()` function to plot a histogram
- Use either `bins` or `binwidth`.
 - The shape of a histogram is sensitive to `bins/binwidth`
 - We should experiment on `bins/binwidth`.

Boxplot with `sns.boxplot()`

Example

hue

```
1 sns.boxplot(data = ti
2             y = 'tota
```

```
1 sns.boxplot(data = ti
2             x = 'time
3             y = 'tota
```

- A boxplot computes a summary of the distribution and then display a specially formatted box.
 - It is useful to visualize how values of a **continuous variable** are distributed across different values of a **categorical variable**.
- We use `sns.boxplot()` function to plot a boxplot.

```
1 sns.boxplot(data = ti
2             x = 'time
3             y = 'tota
4             hue = 'ti
```

```
1 sns.boxplot(data = ti
2             x = 'time
```

```
3      y = 'total'
4      hue = 'sm'
```



- Mapping
 - **hue**: Name of a categorical variable

Scatter plot with `sns.scatterplot()` and `sns.lmplot()`

Example hue lmplot() alpha

lmplot() Example

```
1 sns.scatterplot(data
2                      x = '
3                      y = '
4
```

- Mapping
 - `x`: Name of a continuous variable on the horizontal axis
 - `y`: Name of a continuous variable on the vertical axis
- A scatter plot is used to display the relationship between two continuous variables.
 - We can see co-variation as a pattern in the scattered points.
- We use `sns.scatterplot()` function to plot a scatter plot.

```
1 sns.scatterplot(data = tips,  
2                 x = 'total_bill',  
3                 y = 'tip',  
4                 hue = 'smoker')
```

- To the scatter plot, we can add a **hue-VARIABLE** mapping to display how the relationship between two continuous variables varies by **VARIABLE**.
- Suppose we are interested in the following question:
 - **Q.** Does a smoker and a non-smoker have a difference in tipping behavior?

```
1 sns.lmplot(data = tips,  
2            x = 'total_bill',  
3            y = 'tip')
```

- From the scatter plot, it is often difficult to clearly see the relationship between two continuous variables.
 - **sns.lmplot()** adds a line that fits well into the scattered points.
 - The best fitting line describes the *overall* relationship between two continuous variables.

```
1 sns.scatterplot(da  
2                 x  
3                 y  
4                 al
```

```
1 sns.lmplot(data = tips,  
2           x = 'total_bill',  
3           y = 'tip',  
4           scatter_kws =
```

- In a scatter plot, adding **transparency** with `alpha` helps address many data points on the same location.
 - We can map `alpha` to number between 0 and 1.
 - `alpha = 0`: Full transparency
 - `alpha = 1`: No transparency

```
1 sns.lmplot(data = tips,  
2           x = 'total_bill',  
3           y = 'tip',  
4           hue = 'smoker',  
5           scatter_kws = {'alpha' : 0.5})
```

- To the scatter plot, we can add a `hue-VARIABLE` mapping to display how the relationship between two continuous variables varies by `VARIABLE`.
- Using the fitted lines, let's answer the following question:
 - **Q.** Does a smoker and a non-smoker have a difference in tipping behavior?

Line chart with `sns.lineplot()`

Example

hue

```
1 path_csv = 'https://b
2 dow = pd.read_csv(pat
3 dow['Date'] = dow['Da
4
5 sns.lineplot(data = d
6             x = 'Da
7             y = 'Cl
```

- Mapping
 - **x**: Name of a continuous variable (often time variable) on the horizontal axis
 - **y**: Name of a continuous variable on the vertical axis
- A line chart is used to display the trend in a continuous variable or the change in a continuous variable over a categorical variable.
 - `sns.lineplot()` draws a line by connecting the scattered points in order of the variable on the x-

axis, so that it highlights exactly when changes occur.

```
1 # New data
2 healthexp = (
3     sns.load_dataset("healthexp")
4     .sort_values(["Country", "Year"])
5     .query("Year <= 2020")
6 )
7
8 sns.lineplot(data = healthexp,
9              x = 'Year',
10             y = 'Life_Expectancy',
11             hue = 'Country')
```

- For line charts, we often need to group or connect observations to visualize the number of distinct lines.

Faceting

- Faceting allows us to plot subsets (facets) of our data across subplots.

`FacetGrid(col/row).map()`

`col/row`

```
1 (
2   sns.FacetGrid(
3       data = titanic,
4       row='class',
5       col='sex')
6   .map(sns.histplot, 'age')
7   )
```

1. We create a `.FacetGrid()` object with the data we will be using and define how it will be subset with the `row` and `col` arguments:
 2. We then use the `.map()` method to run a plotting function on each of the subsets, passing along any necessary arguments.
- Some geometric object, such as `sns.lmplot()`, does not work well with `sns.FacetGrid()`.
 - Instead, `sns.lmplot()` supports mapping, `col = 'VAR'` and `row = 'VAR'` to facet the plot.

```
1 sns.lmplot(data = tips,
2           x = 'total_bill', y = 'tip',
```

```
3     hue = 'smoker',  
4     scatter_kws = {'alpha' : 0.5},  
5     col = 'smoker')
```

Visualizing DataFrames with seaborn

Let's do Classwork 12!